

Kalasetu — Backend Database Architecture

Professional Marketplace: Collections, Relationships, Ownership & Development Plan

Purpose: This document defines the MongoDB collection structure for Kalasetu and clearly separates the responsibilities of the two developers. The design assumes one user account can act as both a customer and a professional.

1. Overall System Concept

Kalasetu has two connected sides. The professional side handles becoming a professional, verification, professional profiles, services, availability and professional booking management. The customer side handles service discovery, professional profiles, available slots, booking, cancellation, reviews, reports and blocking.

Important design decision: Do not create separate customer and professional user accounts. The existing **users** collection remains the central identity collection. A user can book services and, after approval, also become a professional.

USER → Professional Application → Professional Profile → Services → Availability → Bookings ← Customer

2. Complete Collection List

#	Collection	Main Owner	Used By	Purpose
1	users	Existing / Shared	Both	Authentication and common user identity
2	service_categories	Partner	Both	Categories used to organize services
3	professional_applications	You	You + Admin	Become-a-professional application
4	professional_profiles	You	Both	Public verified professional information
5	certificates	You	You + Admin + Partner	Professional certificate records/files
6	services	You	Both	Services offered by professionals
7	availability	You	You + Partner	Professional/service time slots
8	bookings	You / Shared	Both	Customer booking lifecycle
9	reviews	Partner	Both	Ratings and reviews after completion
10	reports	Partner	Both + Admin	Report users/services/bookings
11	blocked_users	Partner	Both	User-to-user blocking

3. users

Main owner: Existing / Shared

Suggested fields:

_id | name | username | email | phone | password_hash | is_active | created_at | updated_at

Purpose: This is the central identity collection. Signup and login already use it. Every important feature should reference the user's _id rather than duplicating user identity data.

Relationship / usage: Used by both developers. Your professional application references user_id. The partner's booking flow references the same user_id as customer_id. The same person can therefore be both customer and professional.

4. service_categories

Main owner: Partner

Suggested fields:

_id | name | description | image | is_active | created_at

Purpose: Stores broad service categories such as Photography, Graphic Design, Home Cleaning, Tutoring and Makeup Artist.

Relationship / usage: One category can contain many services. Relationship: service_categories (1) → services (many). The partner owns creation/management, while your service creation uses category_id.

5. professional_applications

Main owner: You

Suggested fields:

_id | user_id | profession | skills[] | experience_years | experience_description | bio | portfolio_url | instagram_url | application_status | rejection_reason | submitted_at | reviewed_at | created_at

Purpose: Stores the application submitted from Become a Professional. The user_id connects the application to the existing account. application_status should initially support PENDING, APPROVED and REJECTED.

Relationship / usage: Relationship: users (1) → professional_applications (0/1 or potentially many historically). Admin reviews the application. Do not create a public professional profile until approval.

6. professional_profiles

Main owner: You

Suggested fields:

_id | user_id | application_id | profession | bio | experience_years | skills[] | location | portfolio_url | instagram_url | verification_status | rating | total_reviews | completed_bookings | badges[] | is_active | created_at | updated_at

Purpose: Represents the public-facing professional profile after approval. It contains information customers need before booking.

Relationship / usage: Relationship: professional_profiles references user_id and the approved application_id. It is read by the partner's service browsing/profile screens. The profile is the parent entity for services, certificates and professional availability.

7. certificates

Main owner: You

Suggested fields:

_id | professional_id | certificate_name | issuing_organization | certificate_file_url | issue_date | expiry_date | verification_status | uploaded_at

Purpose: Stores multiple certificates for a professional. The actual file should be stored through an appropriate file-storage mechanism, while MongoDB stores the file URL/reference and metadata.

Relationship / usage: Relationship: professional_profiles (1) → certificates (many). Certificates can be reviewed by Admin and displayed as verified credentials on the public professional profile.

8. services

Main owner: You

Suggested fields:

_id | professional_id | category_id | title | description | price | duration | location | images[] | skills[] | is_active | created_at | updated_at

Purpose: Stores the actual services offered by verified professionals. Example: Wedding Photography, ■15,000, 4 hours.

Relationship / usage: Relationship: professional_profiles (1) → services (many), and service_categories (1) → services (many). You create/manage services from the professional dashboard. The partner reads/searches them for customers.

9. availability

Main owner: You

Suggested fields:

_id | professional_id | service_id | date | start_time | end_time | status | created_at

Purpose: Stores the slots a professional makes available. Suggested status values: AVAILABLE, BOOKED, BLOCKED.

Relationship / usage: The partner does not create availability. The partner reads it when a customer checks slots. A selected availability slot is referenced by a booking. Cancellation may make a previously booked slot available again, depending on business rules.

10. bookings

Main owner: You / Shared

Suggested fields:

_id | customer_id | professional_id | service_id | availability_id | booking_date | start_time | end_time | price | status | cancellation_reason | created_at | updated_at

Purpose: This is the core transaction collection connecting customer, professional, service and selected time slot. Suggested status values: PENDING, CONFIRMED, CANCELLED, COMPLETED.

Relationship / usage: The partner owns the customer-side booking action, but you should own the core booking backend/business logic because it must validate the professional, service, availability and prevent double booking. Both sides read and update booking information according to permissions.

11. reviews

Main owner: Partner

Suggested fields:

_id | booking_id | customer_id | professional_id | service_id | rating | comment | created_at | updated_at

Purpose: Stores customer feedback after a completed service. Rating can be represented as a numeric value, commonly 1–5.

Relationship / usage: Relationship: bookings (1) → reviews (0/1). A customer should only be allowed to review a completed booking, which helps prevent fake reviews. Reviews can update the professional's displayed rating and review count.

12. reports

Main owner: Partner

Suggested fields:

_id | reporter_id | reported_user_id | service_id | booking_id | reason | description | status | admin_response | created_at

Purpose: Stores complaints or reports such as fake profile, fraud, inappropriate behavior or poor service. Suggested status values: PENDING, REVIEWED, RESOLVED, REJECTED.

Relationship / usage: A report may reference a user, service and/or booking. Admin handles the review and resolution. The partner creates the report from the customer-facing interface.

13. blocked_users

Main owner: Partner

Suggested fields:

`_id | blocker_id | blocked_user_id | created_at`

Purpose: Stores a simple relationship where one user blocks another user without deleting either account.

Relationship / usage: Relationship: users → blocked_users → users. Example: one customer blocks a professional. The application can then prevent or hide relevant future interactions according to the final blocking rules.

14. Complete Relationship Map

The following logical relationship should guide both developers:

USERS

```
■■■ user_id → PROFESSIONAL_APPLICATIONS
■      ■■■ approved → PROFESSIONAL_PROFILES
■
■      ■■■ professional_id → CERTIFICATES
■      ■■■ professional_id → SERVICES
■      ■■■ professional_id → AVAILABILITY
■
■■■ customer_id → BOOKINGS
■      ■■■ professional_id → PROFESSIONAL_PROFILES
■      ■■■ service_id → SERVICES
■      ■■■ availability_id → AVAILABILITY
■
■■■ reporter_id / reported_user_id → REPORTS
■
■■■ blocker_id / blocked_user_id → BLOCKED_USERS
```

SERVICE_CATEGORIES

```
■■■ category_id → SERVICES
```

BOOKINGS

```
■■■ booking_id → REVIEWS
```

15. Responsibility Split Between You and Your Partner

Area	You	Partner
Professional application	Create + manage backend	Not responsible
Professional verification/profile	Create + manage	Read/display
Certificates	Upload + manage + verification support	Read/display verified certificates
Service creation	Create/manage from professional dashboard	Search/display
Service categories	Read/use	Create/manage
Availability	Create/manage	Read/check
Bookings	Core backend/business logic + professional management	Customer-side create/view/cancel
Reviews	Read where needed	Create/manage customer review flow
Reports	Read/admin handling where needed	Create customer reports
Blocked users	Read/enforce where required	Create/manage customer block action

16. Recommended Development Order

1. Verify the existing users collection and authentication/JWT structure.
2. Create professional_applications.
3. Create certificates.

4. Create professional_profiles.
5. Create service_categories and agree with the partner on category fields.
6. Create services.
7. Create availability.
8. Create bookings and finalize booking-status rules.
9. Partner creates reviews.
10. Partner creates reports and blocked_users.
11. Integrate both sides and test the complete booking lifecycle.

17. Recommended Booking Lifecycle

Customer selects a service → selects an available slot → booking is created as PENDING → professional accepts → booking becomes CONFIRMED → service is completed → booking becomes COMPLETED → customer can submit a REVIEW. If cancellation is permitted, the booking becomes CANCELLED and the availability slot can be released again.

18. Collections to Add Later

Do not build these in the first phase: notifications, messages/chat, payments, badges and portfolio as separate collections. Badges and portfolio can initially be simple fields/arrays in the professional profile or service data. Separate collections can be introduced once the core marketplace works.

19. Final Architecture Principle

One user → optional professional application → verified professional profile → services → availability → bookings. The customer side discovers and books those services, while the professional side creates and manages the supply. Bookings are the central connection between both sides.